CS-3006: PARALLEL & DISTRIBUTED COMPUTING

Fault Distributed

Week 08
Advanced MPI Concepts

Spring 2026
Academic Year

Motivation

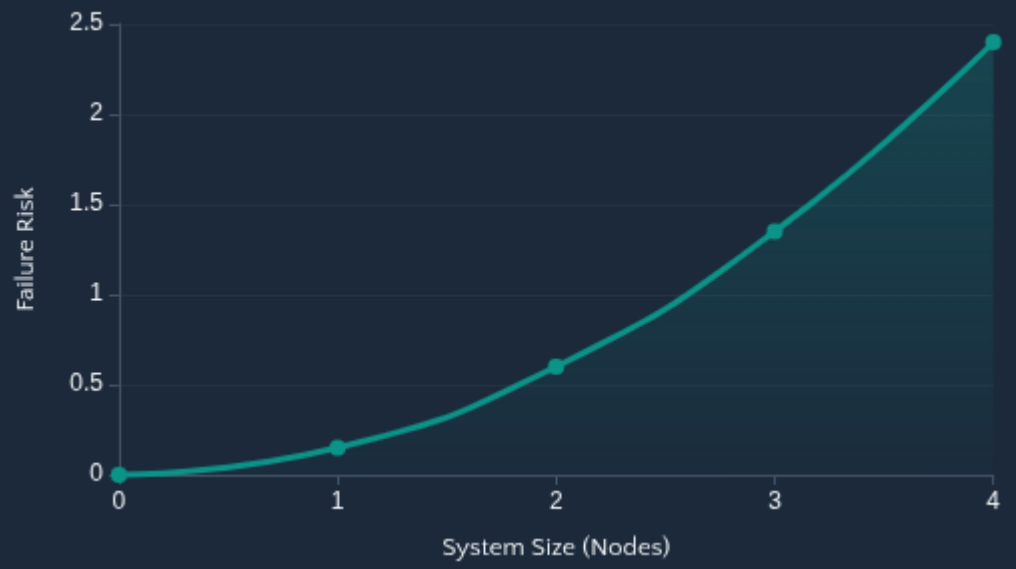
Ideal

$$S = \sum_{i=0}^{N-1} A[i]$$

We assume all P processes stay alive

- Nodes fail, networks stall
- Bigger systems → failures become likely
- Standard MPI assumes perfect execution

Failure



Source: Kshemkalyani & Singhal, 2011

Key Insight: As system size grows, Mean Time Between Failures (MTBF) drops exponentially.

10^3	10^4	10^5+
Small Cluster	Medium HPC	Exascale
Low Risk	Moderate	High Risk

FUNDAMENTALS

Dependability

Standard Taxonomy (Avizienis et al., 2004)

Fault

Physical defect in the system

Examples: Power loss, hardware defect, software bug

activates

Error

Incorrect state in the system

Examples: Corrupt memory, wrong computation

propagates

Failure

Service stops providing correct output

Examples: Job aborts, system crash

Fault

The ability of a system to provide correct service despite the presence of faults. It aims to prevent an error from propagating into a failure.

Key

- **Availability:** Readiness for correct service
- **Reliability:** Continuity of correct service
- **Safety:** Absence of catastrophic consequences
- **Maintainability:** Ease of maintenance and repair

FAILURE ANALYSIS

C o m m o n

What can go wrong? (Kshemkalyani & Singhal, 2011)

R a n k

One process dies unexpectedly, often due to segmentation faults or unhandled exceptions.

Example: Segfault, divide by zero

C o m m u n i c a t i o n

Network partition or message loss disrupts inter-process communication, causing hangs.

Example: Network partition, timeout

N o d e

Hardware fault kills all MPI ranks running on a single machine, affecting multiple processes.

Example: Power outage, CPU failure

S i l e n t

Process continues running but produces wrong results (Byzantine failure) - hardest to detect.

Example: Bit flip, memory corruption

T h e

When something fails, should we: [redacted], [redacted], [redacted], or [redacted] with fewer resources?

COMPARISON

S h a r e d

Attribute	Shared Memory (OpenMP)	Distributed Memory (MPI)
Unit of Failure	The entire process crashes. All threads share a single PID.	Individual process or entire node can fail.
State Scope	Global shared address space. All threads access same memory.	Local address spaces only. Each rank has private memory.
Recovery Mechanism	Process Restart. Must restart entire application.	Rank Exclusion (ULFM) possible. Can continue with fewer ranks.
Complexity	Easier to save state. Single memory image to checkpoint.	Harder. Requires synchronization across distributed state.

K e y

Distributed memory makes fault handling significantly harder because state is spread across multiple nodes, requiring complex coordination for recovery.

STRATEGIES

Main

1 Checkpoint

Save application state to stable storage (disk, SSD, RAM) periodically. On failure, restart from last checkpoint.

- **Pros:** Simple, well-understood
- **Cons:** High I/O overhead

2 Replication

Run redundant copies of critical processes. If one fails, others continue. Common in distributed databases.

- **Pros:** Fast failover
- **Cons:** 2x+ resource cost

3 Communicator

Exclude failed processes from MPI communicator and continue execution with remaining ranks. "Self-healing" approach.

- **Pros:** No restart needed
- **Cons:** Complex implementation

4 Algorithm-Based

Design algorithms that can mathematically correct errors (e.g., checksums, redundant computations). Application-specific.

- **Pros:** Low overhead
- **Cons:** Problem-specific

Big

All techniques try to reduce the effect of failures, but each adds cost (time, storage, or complexity). Choose based on application requirements and failure characteristics.
(Kshemkalyani & Singhal, 2011)

IMPLEMENTATION

M P I

D e f a u l t

MPI defaults to [redacted]: the job aborts immediately on any error.

Problem: No opportunity for recovery or graceful degradation.

H a n d l i n g

To implement custom recovery, change the error handler:

```
// Set error handler to return errors
MPI_Comm_set_errhandler(MPI_COMM_WORLD, MPI_ERRORS_RETURN);

int err = MPI_Send(...);
if (err != MPI_SUCCESS) {
    // Handle failure, initiate recovery
printf("Rank %d: Send failed.\n", rank);
}
```

```
double local_sum = 0.0;
int start_idx = load_checkpoint();

for (int i=start_idx; i
local_sum += data[i];

    // Coordinated Checkpoint
if (i % INTERVAL == 0) {
MPI_Barrier(MPI_COMM_WORLD);
save_to_disk(local_sum, i);
}
}

MPI_Reduce(&local_sum, &global_sum, ...);
```

C h e c k p o i n t

1. **Sync:** Barrier ensures global consistency
2. **Save:** Write local state to stable storage
3. **Restart:** On failure, reload state

Tradeoff: High I/O overhead vs. safety

Note: Standard MPI-3 does not specify how to "fix" a communicator after a process dies. (Gropp et al., 2014)

MPI VULNERABILITY

M P I

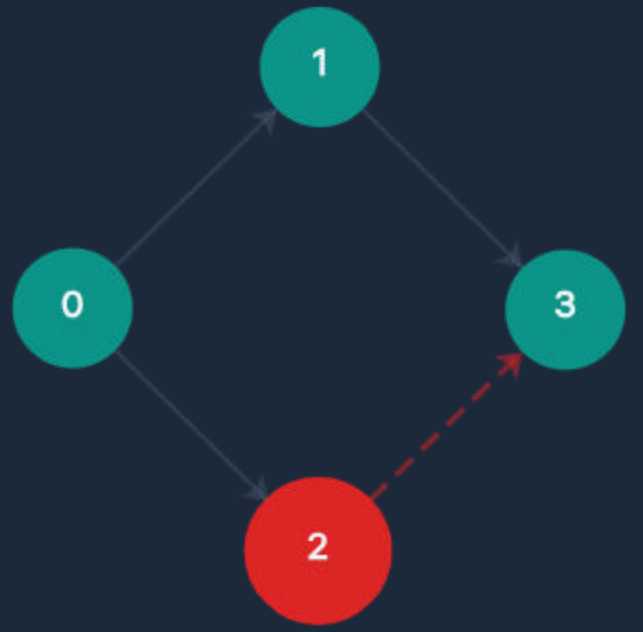
T h e

MPI Collectives (e.g., MPI_Reduce , MPI_Bcast) require **all ranks** in the communicator to participate.

1. If **Rank 2 crashes**...
2. Rank 0 and Rank 1 **wait indefinitely**
3. The job **hangs or aborts**

Result: A single rank failure in a collective operation compromises the entire communicator (Gropp et al., 2014)

C o m m u n i c a t i o n



Healthy Ranks
Ranks 0, 1, 3 waiting

Failed Rank
Rank 2 crashed

ADVANCED MPI

-L e v e l

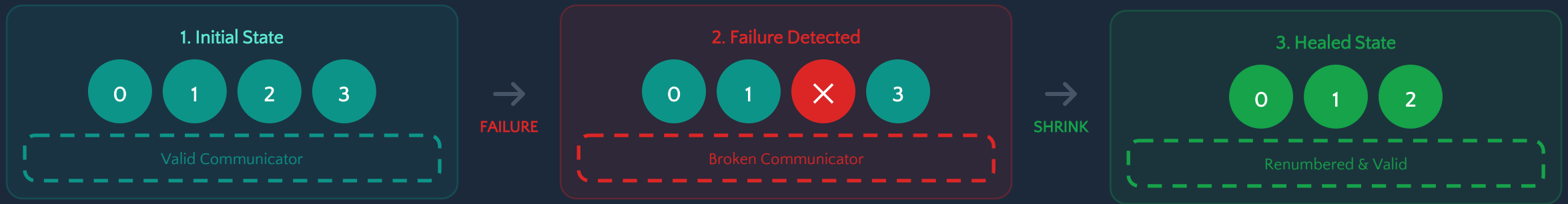
S t a n d a r d

If a rank dies, the communicator becomes invalid. The job aborts immediately with no recovery option.

U L F M

Proposed extension to "heal" the communicator by excluding failed processes and continuing execution.

U L F M



Application Responsibility: Must redistribute work when ranks are excluded (e.g., Rank 1 takes over Rank 2's array chunk). (Bland et al., 2013)

OPENMP

Shared

Failure

Threads Share Single PID

All OpenMP threads belong to one process

One Thread Failure = All Crash

A fault in any thread crashes the entire process

Fault

Application-level

Save shared variables to disk with custom code. Requires programmer effort.

System-level

Tools like DMTCP or BLCR transparently freeze and save process memory.

Transparent to application, easier to use

```
double sum = 0.0;
int start_i = read_checkpoint();

for (int i=start_i; i
        // Parallel computation
        #pragma omp parallel for reduction(+:sum)
for (int j=0; j
sum += data[i+j];
}

        // Implicit barrier here
save_checkpoint(sum, i+CHUNK);
}
```

Why

- ✓ No explicit Barrier (implicit at parallel region end)
- ✓ No distributed consensus needed

⚠ Tradeoff: Process crash kills **all** threads. No partial survival possible.

TOOLS & RESOURCES

Frameworks

Awareness-level knowledge for HPC fault tolerance

Name	Type	Description
SCR (LLNL)	C/R Library	Scalable Checkpoint/Restart with **multilevel checkpointing** (RAM → SSD → PFS). Optimizes I/O performance.
FTI	C/R Library	Fault Tolerance Interface for HPC applications. Easy-to-use API for checkpointing.
ULFM	MPI Ext.	Standard proposal for **MPI communicator repair**. Enables self-healing applications.
DMTCP	System Tool	Distributed MultiThreaded Checkpointing. **Transparent checkpointing** for Linux processes.

These tools address tradeoffs between **I/O overhead** and **recovery speed**. Choose based on your application's checkpoint frequency, data size, and recovery requirements.

KEY TAKEAWAYS

Summary

1 **Terminology**

Understand the progression: Fault → Error → Failure

2 **Failures**

As system size grows, MTBF drops. Fault tolerance is essential for large-scale HPC.

3 **Shared**

Failure kills the entire process. Use C/R with tools like DMTCP.

4 **Distributed**

Failure kills the entire job. Two main solutions:

- ✓ **Checkpoint/Restart** (standard approach)
- ✓ **ULFM** (advanced self-healing)

5 **The**

Resilience costs time, storage, or complexity. Balance based on your application's needs.

Remember: No free lunch in fault tolerance. Every solution has costs.